SYNTH-03: HTTP Monitors

> **Series:** SYNTH — Synthetic Monitoring | **Notebook:** 3 of 6 | **Created:** December 2025 | **Last Updated:** 08/03/2026

Lightweight API and Endpoint Monitoring

This notebook covers HTTP monitors for API health checks, endpoint validation, and multi-step API workflows using the latest Dynatrace platform.

Table of Contents

1. Single Request Monitors
2. Multi-Step HTTP Monitors
3. Authentication
4. Response Validation
5. SSL Certificate Monitoring
6. Analyzing HTTP Results

Prerequisites

- ✅ Access to a Dynatrace environment with Synthetic Monitoring
- ✅ Completed SYNTH-01 Fundamentals
- ✅ API endpoint(s) to monitor

> **Data model:** HTTP results live in `fetch dt.synthetic.events`. Monitor-level rows (`event.type == "http_monitor_execution"`) carry `result.state` and total

> `result.statistics.duration`; per-request rows (`event.type == "http_step_execution"`) carry the timing breakdown (DNS / TCP / TLS / TTFB) and SSL certificate data.

1. HTTP Monitor Overview

HTTP monitors execute lightweight HTTP requests without browser overhead:

HTTP vs Browser Monitors

Aspect	HTTP Monitor	Browser Monitor
Execution	Direct HTTP call	Full browser
Speed	Fast (< 1s typical)	Slower (3-30s)
Resources	Minimal	Chrome instance
JavaScript	Not executed	Fully executed
Frequency	1-60 minutes	5-60 minutes
Cost	Lower	Higher

Use Cases

Scenario	HTTP Monitor Type
API health check	Single request
REST API endpoint	Single request
Auth token + API call	Multi-step
GraphQL queries	Single/Multi-step
Webhook testing	Single request
Certificate expiry	Single request + SSL check

Configuration Path

Dynatrace menu → Synthetic → Create synthetic monitor → Create HTTP monitor

2. Single Request Monitors

Request Configuration

Setting	Description	Example
URL	Full endpoint URL	`https://api.example.com/health`
Method	HTTP verb	GET, POST, PUT, DELETE, PATCH
Headers	Custom headers	`Authorization: Bearer ...`
Body	Request payload	JSON, form data, raw
Timeout	Max wait time	30 seconds (default)

Common HTTP Methods

Method	Use Case	Body
`GET`	Retrieve data	None
`POST`	Create resource	Required
`PUT`	Update resource	Required
`PATCH`	Partial update	Required
`DELETE`	Remove resource	Optional
`HEAD`	Check existence	None
`OPTIONS`	CORS preflight	None

Request Headers

```
Content-Type: application/json
Accept: application/json
Authorization: Bearer &lt;token&gt;
X-API-Key: &lt;key&gt;
User-Agent: Dynatrace Synthetic
```

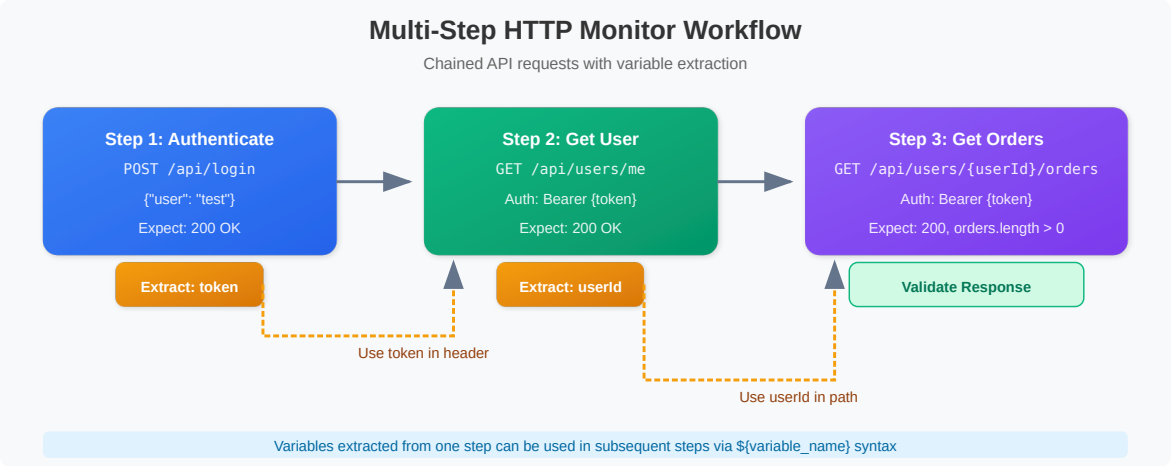
```
// List all HTTP monitors
// PREFERRED -- Smartscape. Classic dt.entity.http_check maps to HTTP_MONITOR;
// id_classic still carries the HTTP_CHECK-* id for joining to unmigrated
queries.
smartscapeNodes "HTTP_MONITOR"
| fields name, id, id_classic, http_monitor.type, enabled, frequency
| sort name asc
| limit 50

// Multi-step HTTP requests are separate nodes (HTTP_MONITOR_STEP) joined to
the monitor
// by an edge, not fields of the monitor record. Steps per monitor:
//
// smartscapeNodes "HTTP_MONITOR_STEP"
// | traverse edgeTypes: {"belongs_to"}, targetTypes: {"HTTP_MONITOR"},
direction: forward
// | summarize steps = count(), by:{monitor = name}
// | sort steps desc

// FALLBACK (classic surface -- still functional):
// fetch dt.entity.http_check
// | fields id, entity.name
// | sort entity.name asc
// | limit 50
```

```
// HTTP monitor execution results (last 24h)
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| fields timestamp,
         monitor = monitor.name,
         location = entityName(dt.entity.synthetic_location),
         state = result.state,
         status_code = result.statistics.response_status_code,
         response_ms = result.statistics.duration / 1ms
| sort timestamp desc
| limit 100
```

3. Multi-Step HTTP Monitors

Chain multiple HTTP requests with data passing between steps:

Multi-Step Workflow Example



Variable Extraction

Source	Syntax	Example
JSON path	`$.data.token`	Extract from JSON body
Response header	`header:X-Request-Id`	Extract from headers
Regex	`token":"([^"]+)`	Pattern matching

Using Variables

Reference extracted variables in subsequent steps:
- URL: `https://api.example.com/users/${userId}`
- Header: `Authorization: Bearer ${token}`
- Body: `{"userId": "${userId}"}`

```
// Multi-step HTTP monitor — per-step performance
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_step_execution"
| summarize {
    avg_duration_ms = avg(result.statistics.duration / 1ms),
    success_rate = countIf(result.status.code == 0) * 100.0 / count(),
    executions = count()
  }, by: {monitor.name, step.name}
| fieldsAdd avg_duration_ms = round(avg_duration_ms, decimals: 2)
| fieldsAdd success_rate = round(success_rate, decimals: 2)
| sort avg_duration_ms desc
| limit 30
```

4. Authentication

Supported Authentication Methods

Method	Configuration	Use Case
Basic Auth	Username/password encoded	Legacy APIs
Bearer Token	Header: `Authorization: Bearer <token>`	OAuth2, JWT
API Key	Header: `X-API-Key: <key>`	Third-party APIs
OAuth2	Token endpoint + credentials	Modern APIs
Client Certificate	mTLS	High-security APIs

Credential Vault

Store sensitive credentials securely:

1. **Settings → Integration → Credential vault**
2. Add credential (username/password, token, certificate)
3. Reference in monitor: `${credentials.vault.myCredential}`

OAuth2 Flow Example

```
Step 1: Get Token
    POST https://auth.example.com/oauth/token
    Body: grant_type=client_credentials
          &amp;client_id=${vault.clientId}
          &amp;client_secret=${vault.clientSecret}
    Extract: access_token

Step 2: API Call
    GET https://api.example.com/data
    Header: Authorization: Bearer ${access_token}
```

5. Response Validation

HTTP Status Validation

Status Range	Meaning	Default Behavior
2xx	Success	Pass
3xx	Redirect	Follow/Pass
4xx	Client Error	Fail
5xx	Server Error	Fail

Content Validation

Type	Description	Example
Contains	Text present	`"status": "ok"`
Not Contains	Text absent	`"error"`
Regex	Pattern match	`"id":\s*\d+`
JSON Path	Value at path	`$.status == "success"`

JSON Path Assertions

```
// Response:
{
  "status": "success",
  "data": {
    "users": [
      {"id": 1, "name": "John"},
      {"id": 2, "name": "Jane"}
    ]
  }
}

// Assertions:
$.status == "success"           // Check status
$.data.users.length &gt; 0         // Array not empty
$.data.users[0].name == "John"  // First user name
```

```
// HTTP status code distribution
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| summarize count = count(), by: {monitor.name,
result.statistics.response_status_code}
| sort count desc
| limit 30
```

```
// Failed HTTP requests with error details
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| filter result.state == "FAIL"
| fields timestamp,
         monitor = monitor.name,
         location = entityName(dt.entity.synthetic_location),
         status = result.status.message,
         status_code = result.status.code,
         detail = result.status.details
| sort timestamp desc
| limit 50
```

6. SSL Certificate Monitoring

HTTP monitors automatically check SSL certificates:

Certificate Checks

Check	Description	Alert Threshold
Validity	Certificate not expired	Configurable days
Chain	Valid certificate chain	Any break
Hostname	Matches request domain	Mismatch
Trust	Issued by trusted CA	Untrusted

Expiration Alerts

Configure alerts for certificates expiring within:
- 30 days (warning)
- 14 days (critical)
- 7 days (emergency)

```
// SSL certificate expiration status (per monitor)
// peer_certificate_expiry_date is carried on http_step_execution records for HTTPS targets
fetch dt.synthetic.events, from: now() - 6h
| filter event.type == "http_step_execution"
| filter isNotNull(result.statistics.peer_certificate_expiry_date)
| summarize { certificate_expiry = takeMax(result.statistics.peer_certificate_expiry_date) }, by: {monitor.name}
| fieldsAdd days_until_expiry = round((certificate_expiry - now()) / 1h / 24, decimals: 1)
| sort days_until_expiry asc
| limit 20
```

7. Analyzing HTTP Results

```
// HTTP monitor availability summary (by monitor)
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| summarize {
    total = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
  }, by: {monitor.name}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals:
2)
| sort availability_pct asc
| limit 30
```

```
// Response time percentiles by monitor (successful executions)
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| filter result.state == "SUCCESS"
| summarize {
    p50 = percentile(result.statistics.duration, 50),
    p95 = percentile(result.statistics.duration, 95),
    p99 = percentile(result.statistics.duration, 99),
    max = max(result.statistics.duration),
    executions = count()
  }, by: {monitor.name}
| fieldsAdd p50_ms = round(p50 / 1ms, decimals: 1),
            p95_ms = round(p95 / 1ms, decimals: 1),
            p99_ms = round(p99 / 1ms, decimals: 1),
            max_ms = round(max / 1ms, decimals: 1)
| fieldsRemove p50, p95, p99, max
| sort p95_ms desc
| limit 20
```

```
// HTTP timing breakdown (DNS, TCP connect, TLS, TTFB) — from step records
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_step_execution"
| filter result.status.code == 0
| summarize {
    avg_dns_ms     = round(avg(result.statistics.host_name_resolution_time) /
1ms, decimals: 2),
    avg_connect_ms = round(avg(result.statistics.tcp_connect_time) / 1ms,
decimals: 2),
    avg_tls_ms     = round(avg(result.statistics.tls_handshake_time) / 1ms,
decimals: 2),
    avg_ttfb_ms    = round(avg(result.statistics.time_to_first_byte) / 1ms,
decimals: 2),
    avg_total_ms   = round(avg(result.statistics.duration) / 1ms, decimals:
2),
    executions = count()
  }, by: {monitor.name, step.name}
| sort avg_total_ms desc
| limit 20
```

```
// Response time trend over time (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter event.type == "http_monitor_execution"
| filter result.state == "SUCCESS"
| makeTimeseries {
    avg_response_ms = avg(result.statistics.duration / 1ms),
    p95_response_ms = percentile(result.statistics.duration / 1ms, 95)
  }, interval: 1h
```

Summary

In this notebook, you learned:

✅ **HTTP monitor types** - Single request vs multi-step
✅ **Request configuration** - Methods, headers, body
✅ **Multi-step workflows** - Variable extraction and chaining
✅ **Authentication** - Basic, Bearer, OAuth2, API keys
✅ **Response validation** - Status codes, content, JSON path
✅ **SSL monitoring** - Certificate expiration via

`result.statistics.peer_certificate_expiry_date`
✅ **Analysis queries** - Availability, percentiles, timing breakdown

---

Next Steps

Continue to **SYNTH-04: Private Locations** to learn about running monitors from your own infrastructure.

References

- Create and configure an HTTP monitor (DT docs)
- HTTP monitor metrics in Synthetic on Grail (DT docs)
- Credential vault (DT docs)
- Synthetic app (DT docs)
